

Topic-1: Packages

A *package* is a collection of related Java entities (such as classes, interfaces, exceptions, errors and enums). Packages are used for:

- Resolving naming conflict of classes by prefixing the class name with a package name. For example, `com.zzz.Circle` and `com.yyy.Circle` are two distinct classes. Although they share the same class name `Circle`, but they belong to two different packages: `com.zzz` and `com.yyy`. These two classes can be used in the same program and distinguished using the *fully-qualified class name* - package name plus class name. This mechanism is called *Namespace Management*

Package Naming Convention

A package name is made up of the *reverse* of the Internet Domain Name (to ensure uniqueness) plus your own organization's internal project name, separated by dots `'.'`. Package names are in lowercase. For example, suppose that your Internet Domain Name is `"zzz.com"`, you can name your package as `"com.zzz.project1.subproject2"`.

The prefix `"java"` and `"javax"` are *reserved* for core Java packages and Java extensions, respectively.

Creating Packages

To make a class as part of a package, you have to include the `package` statement as the first statement in the source file.

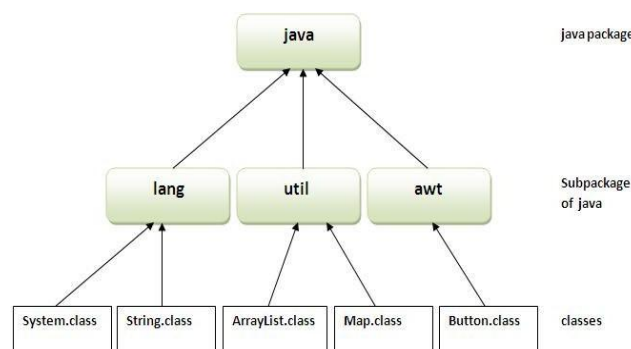
Types of Packages

Package in java can be categorized in two types

- built-in packages
- user-defined packages

Built-in packages

There are many built-in packages such as `java`, `lang`, `awt`, `javax`, `swing`, `net`, `io`, `util`, `sql` etc.



Topic-2: User-defined packages

Step-1 Creating a package

The **package keyword** is used to create a package in java.

```
//save as Simple.java
package mypack;
public class Simple{
    public static void main(String args[]){
        System.out.println("Welcome to package");
    }
}
```

Step-2 compile java package

If you are not using any IDE, you need to follow the syntax given below:

```
javac -d directory javafilename
```

For **example**

```
javac -d . Simple.java
```

The -d switch specifies the destination where to put the generated class file.

Step-3 run java package program

You need to use fully qualified name e.g. mypack.Simple etc to run the class.

To Compile: `javac -d . Simple.java`

To Run: `java mypack.Simple`

Output: Welcome to package

The -d is a switch that tells the compiler where to put the class file i.e. it represents destination. The . represents the current folder.

Step-4 accessing the package

There are three ways to access the package from outside the package.

- `import package.*;`
- `import package.classname;`
- fully qualified name.

Example of package by import package.classname

```
//save by A.java
package pack;
public class A{
    public void msg() {
        System.out.println("Hello");
    }
}
```

```
//save by B.java
package mypack;
import pack.A;
class B{
    public static void main(String args[]){
        A obj = new A();
        obj.msg();
    }
}
```

Output:

Hello

Topic-3: Access Protection in packages

The following table specifies the access protection of packages in java

	Private	Protected	Public	No Modifier
Same class	Yes	Yes	Yes	Yes
Same package subclass	No	Yes	Yes	Yes
Same package non-subclass	No	Yes	Yes	Yes
Different package subclass	No	Yes	Yes	No
Different package non-subclass	No	No	Yes	No

Topic-4: Enumeration in java

- Enum, introduced in Java 5, is a special data type that consists of a set of pre-defined named values separated by commas.
- These named values are also known as elements or enumerators or enum instances.
- Since the values in the enum type are constant, you should always represent them in UPPERCASE letters.

Syntax:

```
enum Variable_Name { VALUE_1, VALUE_2, VALUE_3, ... }
```

Example:

```
class EnumExample1{
//defining the enum inside the class
    public enum Season { WINTER, SPRING, SUMMER, FALL }
//main method
    public static void main(String[] args) {
```

```
//traversing the enum
    for (Season s : Season.values())
        System.out.println(s);
    }
}
```

Output:

```
WINTER
SPRING
SUMMER
FALL
```

Topic-5: java.lang package

- java.lang package contains the classes and interfaces that are fundamental to the design of the Java programming language.
- Wrapper Classes like Integer, Boolean, Character etc are part of lang package.
- We're not supposed to import any class of java.lang package explicitly as being included by default.

Important interfaces and classes in java.lang Package

Interface Summary	
Interface	Description
Appendable	An object to which char sequences and values can be appended.
AutoCloseable	A resource that must be closed when it is no longer needed.
CharSequence	A CharSequence is a readable sequence of char values.
Cloneable	A class implements the Cloneable interface to indicate to the Object.clone() method that it is legal for that method to make a field-for-field copy of instances of that class.
Readable	A Readable is a source of characters.
Runnable	The Runnable interface should be implemented by any class whose instances are intended to be executed by a thread.

Class Summary	
Class	Description
Boolean	The Boolean class wraps a value of the primitive type boolean in an object.
Byte	The Byte class wraps a value of primitive type byte in an object.
Character	The Character class wraps a value of the primitive type char in an object.
ClassLoader	A class loader is an object that is responsible for loading classes.
Double	The Double class wraps a value of the primitive type double in an object.
Float	The Float class wraps a value of primitive type float in an object.
Integer	The Integer class wraps a value of the primitive type int in an object.
Long	The Long class wraps a value of the primitive type long in an object.
Math	The class Math contains methods for performing basic numeric operations such as the elementary exponential, logarithm, square root, and trigonometric functions.
Number	The abstract class Number is the superclass of classes BigDecimal, BigInteger, Byte, Double, Float, Integer, Long, and Short.
Object	Class Object is the root of the class hierarchy.
Short	The Short class wraps a value of primitive type short in an object.
String	The String class represents character strings.

StringBuffer	A thread-safe, mutable sequence of characters.
StringBuilder	A mutable sequence of characters.
System	The System class contains several useful class fields and methods.
Thread	A thread is a thread of execution in a program.
Throwable	The Throwable class is the superclass of all errors and exceptions in the Java language.
Void	The Void class is an uninstantiable placeholder class to hold a reference to the Class object representing the Java keyword void.

Topic-6: java.lang.Object class

The **Object class** is the parent class of all the classes in java by default. In other words, it is the topmost class of java.

The Object class is beneficial if you want to refer any object whose type you don't know. Notice that parent class reference variable can refer the child class object, known as upcasting.

The Object class provides some common behaviors to all the objects such as object can be compared, object can be cloned, object can be notified etc

Methods of Object class

Method	Description
public final Class getClass()	returns the Class class object of this object. The Class class can further be used to get the metadata of this class.
public int hashCode()	returns the hashcode number for this object.
public boolean equals(Object obj)	compares the given object to this object.
protected Object clone() throws CloneNotSupportedException	creates and returns the exact copy (clone) of this object.
public String toString()	returns the string representation of this object.

<code>public final void notify()</code>	wakes up single thread, waiting on this object's monitor.
<code>public final void notifyAll()</code>	wakes up all the threads, waiting on this object's monitor.
<code>public final void wait(long timeout)throws InterruptedException</code>	causes the current thread to wait for the specified milliseconds, until another thread notifies (invokes <code>notify()</code> or <code>notifyAll()</code> method).
<code>public final void wait(long timeout,int nanos)throws InterruptedException</code>	causes the current thread to wait for the specified milliseconds and nanoseconds, until another thread notifies (invokes <code>notify()</code> or <code>notifyAll()</code> method).
<code>public final void wait()throws InterruptedException</code>	causes the current thread to wait, until another thread notifies (invokes <code>notify()</code> or <code>notifyAll()</code> method).
<code>protected void finalize()throws Throwable</code>	is invoked by the garbage collector before object is being garbage collected.

Example:

```
public class ObjectDemo {
    int car_no;

    ObjectDemo(int car_no){
        this.car_no=car_no;
    }

    // Driver code
    public static void main(String args[])
    {
        ObjectDemo s = new ObjectDemo(6341);
        ObjectDemo t = new ObjectDemo(8122);

        System.out.println(s);
        System.out.println(t);
        System.out.println(s.hashCode());
        System.out.println(t.hashCode());
    }
}
```

Output:

```
ObjectDemo@2a84aee7
ObjectDemo@a09ee92
713338599
168423058
```

Topic-7: java.lang.Math Class

The **java.lang.Math** class contains methods for performing basic numeric operations such as the elementary exponential, logarithm, square root, and trigonometric functions.

Basic Math methods

Method	Description
<u>Math.abs()</u>	It will return the Absolute value of the given value.
<u>Math.max()</u>	It returns the Largest of two values.
<u>Math.min()</u>	It is used to return the Smallest of two values.
<u>Math.round()</u>	It is used to round of the decimal numbers to the nearest value.
<u>Math.sqrt()</u>	It is used to return the square root of a number.
<u>Math.cbrt()</u>	It is used to return the cube root of a number.
<u>Math.pow()</u>	It returns the value of first argument raised to the power to second argument.
<u>Math.ceil()</u>	It is used to find the smallest integer value that is greater than or equal to the argument or mathematical integer.
<u>Math.floor()</u>	It is used to find the largest integer value which is less than or equal to the argument and is equal to the mathematical integer of a double value.
<u>Math.random()</u>	It returns a double value with a positive sign, greater than or equal to 0.0 and less than 1.0.
<u>Math.hypot()</u>	It returns $\sqrt{x^2+y^2}$ without intermediate overflow or underflow.

Example:

```
public class MathDemo {  
  
    public static void main(String[] args) {  
  
        // get two double numbers  
        double x = 60984.1;  
        double y = -497.99;  
  
        // get the natural logarithm for x  
        System.out.println("Math.log(" + x + ")=" + Math.log(x));  
    }  
}
```

```

// get the natural logarithm for y
System.out.println("Math.log(" + y + ")=" + Math.log(y));

// get the max value
System.out.println("Math.max(" + x + ", y" + ")=" +
Math.max(x, y));

// get the min value
System.out.println("Math.min(" + x + ", y" + ")=" +
Math.min(x, y));

}
}

```

Output:

```

Math.log(60984.1)=11.018368453441132
Math.log(-497.99)=NaN
Math.max(60984.1, y)=60984.1
Math.min(60984.1, y)=-497.99

```

Topic-8: Wrapper Classes in java

- In Java, wrapper classes are used to convert primitive data types into objects.
- Each primitive type has a corresponding wrapper class in the `java.lang` package. Here's a quick overview of the primitive types and their wrapper classes:

Primitive Type Wrapper Class

<code>int</code>	<code>Integer</code>
<code>boolean</code>	<code>Boolean</code>
<code>char</code>	<code>Character</code>
<code>byte</code>	<code>Byte</code>
<code>short</code>	<code>Short</code>
<code>long</code>	<code>Long</code>
<code>float</code>	<code>Float</code>
<code>double</code>	<code>Double</code>

Advantages of Wrapper Class in Java

- **Provide an Object representation of primitive data types:** Wrapper classes provide an object representation of primitive data types, which allows developers to use primitives as objects in their code.
- **Facilitate type conversion:** Wrapper classes can be used to convert primitive data types to and from String representation, making it easy to store primitives in collections or pass them as method arguments.
- **Help in implementing Autoboxing and Unboxing:** Java provides a feature called Autoboxing and Unboxing, which automatically converts between primitive and wrapper class objects, simplifying the code and reducing the number of explicit type conversions required.

Disadvantages of Wrapper Class in Java

- **Performance Overhead:** The process of converting primitive data types to and from Wrapper classes can result in performance overhead, especially in large-scale applications where this conversion takes place frequently.
- **Increased Memory Usage:** Wrapper classes consume more memory than primitive data types, as they are objects and contain additional information like type information, methods, etc.
- **Immutable:** Wrapper classes are immutable, meaning their values cannot be changed once they are created. This can be limiting in certain situations where it is necessary to modify the value of a primitive data type.

Topic-9: Autoboxing and Unboxing in java

Autoboxing:

- Autoboxing in Java refers to the automatic conversion of primitive data types to their corresponding wrapper classes.
- This feature was introduced in Java 5 and makes it easier to use primitive values in situations where only objects are accepted, such as in collections and method parameters.
- Prior to Java 5, converting a primitive value to its corresponding wrapper class in java required manual conversion, using the constructor of the wrapper class.

Unboxing:

- The process of converting a wrapper class in java's object back into its corresponding primitive data type is called "unboxing."

Example:

```
//Java Program to convert all primitives into its corresponding
//wrapper objects and vice-versa
public class WrapperExample{
    public static void main(String args[]){
        byte b=10;
        short s=20;
        int i=30;
        long l=40;
        float f=50.0F;
        double d=60.0D;
        char c='a';
        boolean b2=true;

        //Autoboxing: Converting primitives into objects
        Byte byteobj=b;
        Short shortobj=s;
        Integer intobj=i;
        Long longobj=l;
        Float floatobj=f;
        Double doubleobj=d;
        Character charobj=c;
        Boolean boolobj=b2;
```

```
//Printing objects
    System.out.println("---Printing object values---");
    System.out.println("Byte object: "+byteobj);
    System.out.println("Short object: "+shortobj);
    System.out.println("Integer object: "+intobj);
    System.out.println("Long object: "+longobj);
    System.out.println("Float object: "+floatobj);
    System.out.println("Double object: "+doubleobj);
    System.out.println("Character object: "+charobj);
    System.out.println("Boolean object: "+boolobj);

//Unboxing: Converting Objects to Primitives
    byte bytevalue=byteobj;
    short shortvalue=shortobj;
    int intvalue=intobj;
    long longvalue=longobj;
    float floatvalue=floatobj;
    double doublevalue=doubleobj;
    char charvalue=charobj;
    boolean boolvalue=boolobj;

//Printing primitives
    System.out.println("---Printing primitive values---");
    System.out.println("byte value: "+bytevalue);
    System.out.println("short value: "+shortvalue);
    System.out.println("int value: "+intvalue);
    System.out.println("long value: "+longvalue);
    System.out.println("float value: "+floatvalue);
    System.out.println("double value: "+doublevalue);
    System.out.println("char value: "+charvalue);
    System.out.println("boolean value: "+boolvalue);
}
}
```

Output:

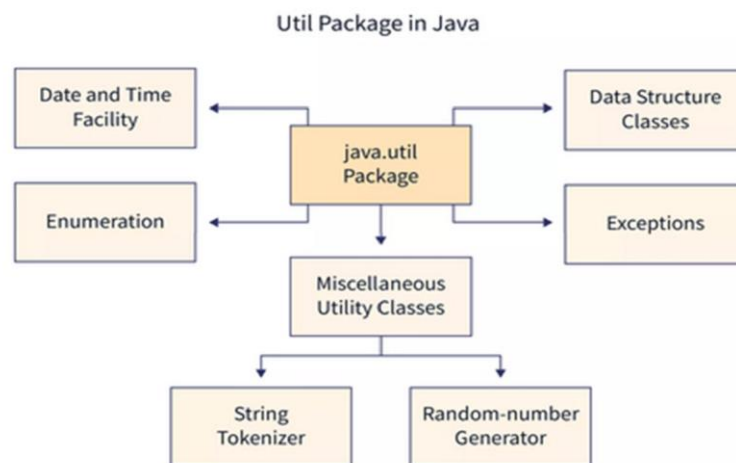
```
---Printing object values---
Byte object: 10
Short object: 20
Integer object: 30
Long object: 40
Float object: 50.0
Double object: 60.0
Character object: a
Boolean object: true
---Printing primitive values---
byte value: 10
short value: 20
int value: 30
long value: 40
float value: 50.0
double value: 60.0
char value: a
boolean value: true
```

Topic-10: java.util package

The java.util package contains various classes and methods that perform common tasks such as **string tokenization**, **date and time formatting**, Java Collections implementation, etc.

The java.util package in Java consists of several components that provide basic necessities to the programmer. These components include:

- **Data Structure Classes:** The **java.util** package contains several pre-written data structures like Dictionary, Stack, LinkedList, etc. that can be used directly in the program using the **import** statements.
- **Date and Time Facility:** The **java.util** package provides several date and time-related classes that can be used to create and manipulate date and time values in a program.
- **Enumeration:** It provides a special interface known as enumeration(enum for short) that can be used to iterate through a set of values.
- **Exceptions:** It provides classes to handle commonly occurring exceptions in a Java program.
- **Miscellaneous Utility Classes:** The **java.util** package contains essential utilities such as the string tokenizer and random-number generator.



java.util Package Interfaces

Interface	Description
Collections	Root interface for the Collections API
Comparator	Provides sorting logic
Deque	Implements Deque data structure
Enumeration	Produces enumeration objects
Iterator	Provides iteration logic
List	Implements an ordered collection (Sequence)
Map	Implements Map data structure (key-value pairs)

Interface	Description
Queue	Implements Queue data structure
Set	Produces a collection that contains no duplicate elements
SortedSet	Produces a set that remembers the total ordering

java.util Package Classes

Class	Description
Collections	Provides methods to represent and manage collections
Formatter	An interpreter for format strings
Scanner	Text scanner used to take user inputs
Arrays	Provides methods for array manipulation
LinkedList	Implements Doubly-linked list data structure
HashMap	Implements Map data structure using Hash tables
TreeMap	Implements Red-Black tree data structure
Stack	Implements last-in-first-out (LIFO) data structure
PriorityQueue	Implements unbounded priority queue data structure
Date	Represents a specific instance of time
Calendar	Provides methods to manipulate calendar fields
Random	Used to generate a stream of pseudorandom numbers.
StringTokenizer	Used to break a string into tokens
Timer, TimerTask	Used by threads to schedule tasks
UUID	Represents an immutable universally unique identifier

Topic-11: java.util.Formatter class

- Formatter is a built-in class in java used for layout justification and alignment, common formats for numeric, string, and date/time data, and locale-specific output in java programming.
- The Formatter class is defined as final class inside the java.util package.

Some Methods of Formatter Class

Methods with Description
Formatter format(String format, Object... args) It writes a formatted string to the invoking object's destination using the specified format string and arguments.
void flush() It flushes the invoking formatter.
String toString()

Methods with Description

It converts the invoking object to string.

void close()

It closes the invoking formatter.

Example:

```
import java.util.Formatter;

public class FormatterExample {
    public static void main(String[] args) {
        // Create a Formatter instance
        StringBuilder sb = new StringBuilder();
        Formatter formatter = new Formatter(sb);

        // Formatting strings
        String name = "Rajesh";
        int age = 40;
        double salary = 65900.67;

        // Using the format method to create a formatted string
        formatter.format("Name: %s, Age: %d, Salary: %.2f", name,
age, salary);

        // Output the formatted string
        System.out.println(sb.toString());

        // Closing the formatter
        formatter.close();
    }
}
```

Output:

Name: Rajesh, Age: 40, Salary: 65900.67

Topic-12: java.util.Random class

- The java.util.Random class is used to generate **pseudorandom numbers**. The methods implemented by this class are used to generate different random data types such as int, double, and float.
- There are two major types of random numbers, **statistically random and pseudorandom numbers**.
- The statistically random numbers (like simple random numbers in maths), are a set of values that do not have a recognizable pattern.
 - For example, rolling a dice 10 times will generate a random number each time that does not have any recognizable pattern.

What are Pseudo Random Numbers?

- It is a set of values that are statistically random but have a **known starting point**.
- In Java, each pseudorandom number is produced through an algorithm. So typically a cycle is repeated every time `java.util.Random` generates a random number.
- A random number generator in Java will produce statistically random numbers but with a known starting point, generated by an algorithm. That makes those values pseudorandom.

S.No	Method	Description
1.	<code>nextBytes(byte[])</code>	This method generates random bytes and stores them in a user-supplied byte array. This method throws a Null Pointer Exception when the parameter <i>arr</i> is empty.
3.	<code>nextInt()</code>	This method returns the next pseudorandom, which is a uniformly distributed int value from this random number generator's sequence. The basic idea behind the <code>nextInt()</code> method, is that one int value is pseudorandomly generated and returned.
4.	<code>nextInt(int)</code>	The parameters here are bound specifies the range till which the number can be generated. The range of the generated number is from [0, bound-1]. It can generate an int value between 0(inclusive) and bound(exclusive)
5.	<code>nextLong()</code>	This method returns the next pseudorandom which is a uniformly distributed long value from this random number generator's sequence.
6.	<code>nextBoolean()</code>	This method returns either <code>True</code> or <code>False</code> . The values <code>True</code> and <code>False</code> are generated with (approximately) equal probability.
7.	<code>nextFloat()</code>	This method returns the next pseudorandom float value, which is a uniformly distributed float value between 0.0 which is included, 1.0 excluded, from the random number generator's sequence.
8.	<code>nextDouble()</code>	This method returns the next pseudorandom double value, which is a uniformly distributed double value between 0.0 which is included, and 1.0 which is not included, from this random number generator's sequence.

Example:

```
import java.util.Random;

public class RandomClassExample {

    public static void main(String[] args) {

        // create random object
        Random randomNumbers = new Random();

        System.out.println("-----Random Boolean-----" );
        /*
        * Returns the next pseudo random boolean value which
        * may be used in a toss for a match
        */
        boolean value = randomNumbers.nextBoolean();
        System.out.println("The random boolean value is: " +
value);
```

```

        /*
        * Returns the next pseudo random integer value between 0
and 5
        * because if we use '6' then, it will give random
numbers from 0 to 6
        * hence incrementing it by 1 you can use it as a result
of a dice roll
        */
        System.out.println("\n-----Random Integer-----" );
        System.out.println("Die Roll: " +
(randomNumbers.nextInt(6)+1) );
        System.out.println("Die Roll: " +
(randomNumbers.nextInt(6)+1) );
        System.out.println("Die Roll: " +
(randomNumbers.nextInt(6)+1) );
        System.out.println("Die Roll: " +
(randomNumbers.nextInt(6)+1) );
        System.out.println("Die Roll: " +
(randomNumbers.nextInt(6)+1) );

        // return the next pseudo random long value
        Long val = randomNumbers.nextLong();
        System.out.println("\n-----Random Long-----" );
        System.out.println("Random Long value: " + val);

    }
}

```

Output:

```

-----Random Boolean-----
The random boolean value is: false

-----Random Integer-----
Die Roll: 2
Die Roll: 6
Die Roll: 3
Die Roll: 4
Die Roll: 4

-----Random Long-----
Random Long value: 856037252185948712

```

Topic-13: java.time Package & Formatting Date/Time in java**java.time package:**

- The `java.time` package, introduced in Java 8, provides a comprehensive and flexible date and time API that overcomes many limitations of the older `java.util.Date` and `java.util.Calendar` classes.
- The `java.time` package is based on the ISO-8601 standard and is designed to handle date and time in a clear and consistent way.

Important Classes in java.time package

1. **LocalDate**: Represents a date without time (e.g., 2023-10-17).
2. **LocalTime**: Represents a time without a date (e.g., 14:30:00).
3. **LocalDateTime**: Combines both date and time without a timezone (e.g., 2023-10-17T14:30:00).
4. **ZonedDateTime**: Represents a date and time with a timezone (e.g., 2023-10-17T14:30:00Z[Europe/Paris]).
5. **Instant**: Represents a specific moment on the timeline (e.g., a point in time with nanosecond precision).
6. **Duration**: Represents a duration of time (e.g., between two moments).
7. **Period**: Represents a date-based amount of time in years, months, and days.

Formatting Date/Time in java

Example:

```
import java.time.LocalDate;
import java.time.LocalTime;
import java.time.LocalDateTime;
import java.time.ZonedDateTime;
import java.time.format.DateTimeFormatter;

public class JavaTimeExample {
    public static void main(String[] args) {
        // Get the current date
        LocalDate today = LocalDate.now();
        System.out.println("Today's Date: " + today);

        // Get the current time
        LocalTime now = LocalTime.now();
        System.out.println("Current Time: " + now);

        // Get the current date and time
        LocalDateTime currentDateTime = LocalDateTime.now();
        System.out.println("Current Date and Time: " +
currentDateTime);

        // Get the current date and time with a timezone
        ZonedDateTime zonedDateTime = ZonedDateTime.now();
        System.out.println("Current Zoned Date and Time: " +
zonedDateTime);

        // Formatting dates
        DateTimeFormatter formatter =
DateTimeFormatter.ofPattern("dd-MM-yyyy HH:mm:ss");
        String formattedDateTime =
currentDateTime.format(formatter);
        System.out.println("Formatted Date and Time: " +
formattedDateTime);
    }
}
```

```

        // Parsing a date string
        String dateStr = "17-10-2023 14:30:00";
        LocalDateTime parsedDateTime = LocalDateTime.parse(dateStr,
formatter);
        System.out.println("Parsed Date and Time: " +
parsedDateTime);
    }
}

```

Output:

```

Today's Date: 2024-10-17
Current Time: 15:37:08.611204400
Current Date and Time: 2024-10-17T15:37:08.611204400
Current Zoned Date and Time: 2024-10-
17T15:37:08.611204400+05:30[Asia/Calcutta]
Formatted Date and Time: 17-10-2024 15:37:08
Parsed Date and Time: 2023-10-17T14:30

```

Topic-14: java.time. TemporalAdjusters class

- TemporalAdjusters class in Java, part of the java.time package, provides a set of predefined implementations of the TemporalAdjuster interface.
- It helps you manipulate dates and times with common adjustments, such as **finding the first day of the month, the last day of the month, or the next specific day of the week.**

Commonly Used methods of TemporalAdjusters class

- **firstDayOfMonth():** Adjusts the date to the first day of the month.
- **lastDayOfMonth():** Adjusts the date to the last day of the month.
- **next(DayOfWeek dayOfWeek):** Adjusts the date to the next occurrence of the specified day of the week.
- **previous(DayOfWeek dayOfWeek):** Adjusts the date to the previous occurrence of the specified day of the week.
- **firstInMonth(DayOfWeek dayOfWeek):** Adjusts the date to the first occurrence of the specified day of the week in the month.

Example:

```

import java.time.LocalDate;
import java.time.DayOfWeek;
import java.time.temporal.TemporalAdjusters;

public class TemporalAdjustersExample {
    public static void main(String[] args) {
        // Current date
        LocalDate today = LocalDate.now();
        System.out.println("Today's Date: " + today);

        // First day of the month
        LocalDate firstDayOfMonth =
today.with(TemporalAdjusters.firstDayOfMonth());

```

```
        System.out.println("First Day of the Month: " +
firstDayOfMonth);

        // Last day of the month
        LocalDate lastDayOfMonth =
today.with(TemporalAdjusters.lastDayOfMonth());
        System.out.println("Last Day of the Month: " +
lastDayOfMonth);

        // Next Monday
        LocalDate nextMonday =
today.with(TemporalAdjusters.next(DayOfWeek.MONDAY));
        System.out.println("Next Monday: " + nextMonday);

        // Previous Friday
        LocalDate previousFriday =
today.with(TemporalAdjusters.previous(DayOfWeek.FRIDAY));
        System.out.println("Previous Friday: " + previousFriday);

        // First Tuesday of the current month
        LocalDate firstSundayOfMonth =
today.with(TemporalAdjusters.firstInMonth(DayOfWeek.SUNDAY));
        System.out.println("First Sunday of the Month: " +
firstSundayOfMonth);
    }
}
```

Output:

```
Today's Date: 2024-10-17
First Day of the Month: 2024-10-01
Last Day of the Month: 2024-10-31
Next Monday: 2024-10-21
Previous Friday: 2024-10-11
First Sunday of the Month: 2024-10-06
```